Lesson 6.2 — Team Dynamics in the AI Era

Lesson 6.2 — Team Dynamics in the AI Era

> **AI did not just change how individuals code. It changed how teams collaborate.** This lesson is the cultural answer set — the questions interviewers ask increasingly often and few candidates rehearse for.

By the end of this lesson:

- You will know how AI changes pair programming, code review, ownership, and onboarding.
- You will have prepared answers to the seven cultural questions most likely to come up.
- You will know how to talk about mentoring juniors on an AI-using team.
- You will have the vocabulary to describe a healthy AI-using team — which signals you have been on one or built one.

---

1. Why this lesson matters

For the first time in interview history, technical questions pass through a cultural filter that is *also* technical. A team that uses AI well looks different from one that uses AI badly, and interviewers know it. They want to hire people who help build the first kind.

The cultural questions are not "soft." They have right and wrong answers. **This lesson is those answers.**

---

2. The seven cultural questions

You will encounter at least three of these in any senior loop. Drill all seven.

1. *"How does AI change pair programming on your team?"*
2. *"How do you review code differently when you suspect AI helped write it?"*
3. *"How do you onboard a junior on a team that uses AI heavily?"*
4. *"How do you handle a teammate whose AI-generated PRs are creating tech debt?"*
5. *"What's your team's policy on AI-generated tests / docs / commits?"*
6. *"How do you balance speed with skill development for newer engineers?"*
7. *"What does a healthy AI-using engineering culture look like?"*

Each gets its own section.

---

3. Pair programming with AI in the room

What changed

Old pair programming: two engineers, one keyboard, taking turns driving and navigating. Both think; one types.

New pair programming: two engineers, *and* an LLM. Three voices in the conversation. The LLM doesn't take turns; it's always available. The risk is that it becomes the loudest voice.

The 30-second answer

> *"Pair programming with AI in the room takes deliberate discipline. The temptation is to outsource the friction — the moments where two engineers would normally argue about an approach get short-circuited by 'let's just ask Claude.' That's where the value of pairing dies. The way I run pair sessions: we form a hypothesis ourselves before any prompt, we use the AI as a shared sounding board not as the decider, and we narrate our disagreements out loud. The AI is the third pair member, not the senior architect. Pairing well with AI is harder than pairing without, not easier."*

Why this answer lands

It identifies the failure mode (outsourcing friction), names a discipline (form hypothesis first), and ends with a memorable line (*"harder, not easier"*). Drill it.

---

4. Code review that respects AI

What changed

A reviewer can no longer assume the diff was hand-typed. Every PR could be AI-generated, AI-iterated, AI-reviewed before submission. The review needs to handle both cases.

The 30-second answer

> *"My review starts with: assume good intent, but read more carefully than I would have five years ago. The failure modes of AI-assisted PRs are different from purely-human ones — fabricated APIs, plausible patterns that don't match our codebase, helpful refactors of code the author didn't ask about. So my review checklist has AI-specific items now. I also try to leave the author room to explain their process. Sometimes the diff that looks suspicious is actually well-considered, and the author had to override the AI to land it. I want to hear that story before I assume the worst."*

The senior follow-up move

> *"I never write 'is this AI-generated?' in a review comment. It puts the author on the defensive. If something looks off, I ask 'can you walk me through the reasoning here?' The answer tells me everything I need to know — about the AI's involvement and about the author's understanding. The conversation is the review."*

That paragraph is staff-grade. **The principle** *"the conversation is the review"* **is worth memorizing.**

---

5. Onboarding juniors

The hard truth

A junior on a modern team has a hard tradeoff: AI accelerates their feature output but slows the development of their *intuitions*. They ship faster than they can reason. The risk is a class of engineer who looks productive on metrics but cannot debug, cannot design, cannot reason about systems.

The 30-second answer

> *"My approach to junior onboarding has shifted. The first month, I have them do at least one task per week without AI. Not as a punishment — as a deliberate exercise in building reasoning muscles that AI can't build for them. We pair on those tasks. By month three I want them comfortable with AI tooling, but only after they have hand-typed enough code to feel the rhythm of the language. The goal is the same as it always was — produce engineers who can think — but the curriculum is different now because the temptation to bypass thinking is greater."*

The interview soundbite

> *"AI raises the floor for juniors. Our job is to ensure it doesn't lower their ceiling."*

That sentence — write it down. Use it once per loop. **It will be repeated by the interviewer to the hiring committee.**

---

6. Handling teammate friction over AI use

The setup question

> *"You have a teammate who consistently submits AI-generated PRs that you have to clean up. How do you handle it?"*

The structured answer

> *"Three layers of response, escalating only as needed. Layer one: a private, peer-level conversation. 'I have noticed a pattern — here are two recent PRs and what*

I caught. I think the AI is slipping past your review. Want to pair on the next one and I'll show you the catches I make?' Most engineers respond well to a peer offering to help.

Layer two, if the pattern continues: a documented version of the same conversation, possibly involving a tech lead. Same content, less ambiguity about whether the message landed.

Layer three, only if the previous two fail and the situation is hurting the team: manager involvement. By that point it's a performance conversation, not a peer one.

I always start at peer level. The conversation is about review skill, not tool choice. People rarely respond well to 'stop using AI'; they often respond well to 'let me show you what I look for.' "

That answer demonstrates: peer-first instinct, escalation discipline, kindness, and a structured approach to a hard conversation. **All four are senior signals.**

---

7. Team policy on AI-generated content

The 30-second answer

"My take on policy is: be specific about what counts as AI-generated. We don't audit everyone's editor. What we do *require: any AI-assisted PR is marked in the description, every line is the author's responsibility regardless of source, AI-generated tests must be reviewed for whether they actually test the new behavior, and AI-generated documentation must be edited for accuracy because the model will invent features that don't exist. The policy is short, but it pushes accountability back to the human and creates a record of when AI was used heavily."*

The "should AI write commit messages?" question

This shows up. Have an answer:

"AI writes a fine first draft. The author has to make sure the message reflects intent — not just what changed, but why. Why is the part the AI doesn't know. So:

AI assists, author owns. Same rule as the code itself."

---

8. Balancing speed and skill

The 30-second answer

> *"There is a real tension between team velocity and individual development. AI lets a junior ship features faster than they can fully reason about them. The team benefits today; the engineer's career suffers tomorrow. My approach is to budget* deliberate practice *into the team — code reading clubs, debugging exercises without AI, design reviews where the junior has to defend their choices. None of those produce features that month, but they produce engineers who'll be senior next year. The team that only optimizes for current velocity hires juniors who never become seniors. That's a hiring problem two years out, and it's avoidable."*

Why this lands with hiring managers

Because it's the conversation they're having internally. They're worried about the same thing you just articulated. **You sound like the person who solves their problem.**

---

9. What a healthy AI-using engineering culture looks like

When an interviewer asks you to describe a healthy AI-using team — or a sick one — you should have a vivid, observable list.

A healthy team

- Authors mark when a PR is heavily AI-assisted, without shame.
- Reviewers ask "walk me through the reasoning" rather than "did the AI write this?"
- Designs are still discussed in design docs by humans before code is written.
- One pair-programming session per sprint runs without AI, by choice.
- Postmortems include "what was the AI's role and what was ours?" without blame.
- Juniors are paired with seniors on real work, not isolated with AI as their only collaborator.

- The team has a shared, written prompt library and treats it as code.

A sick team

- AI use is hidden because the team treats it as cheating.
- PRs ship that no human can fully explain.
- Tests pass mysteriously after AI involvement and nobody asks why.
- Postmortems blame the model.
- Juniors do not get mentorship because seniors think AI provides it.
- Design discussions happen inside individual prompts, not in shared docs.
- There is no policy, and the absence of policy means inconsistent practice.

When asked about culture, paint one of these pictures concretely.

---

10. The "what would you do in your first 90 days?" question

This question is increasingly framed in AI terms. *"How would you approach a team that has not yet adopted AI tools?"* Or *"How would you clean up a team where AI use has gotten messy?"*

The 30-second answer (greenfield team)

> *"In a team that's not using AI yet, I would not lead with adoption. I would lead with a use-case audit — what tasks are eating the team's time that the right tooling could help with? Then propose a small, time-boxed pilot for one task category, with explicit success criteria. If it works, expand. If not, we learn. Adoption is a series of validated bets, not a top-down mandate."*

The 30-second answer (messy team)

> *"In a team where AI use has gotten messy, I'd start by listening. What are people doing? What feels uncomfortable? Where are PRs landing that nobody fully owns? From the listening, write a one-page team policy — short, focused on accountability and review, not tool choice. Then pair the policy with one new ritual: a weekly 30-minute review club where we read recent PRs together, AI-touched and not, and discuss. Most cultural fixes are observation rituals more*

than rules."

Both answers are calm, structured, and people-first. They project: this person can land in a team without breaking it.

---

11. The Q&A trap — when interviewers ask you about *your* team's AI use

If you're employed and the interviewer asks: *"Tell me about how your current team uses AI."* — be careful.

The right framing

- Be honest but generous.
- Don't trash your current team.
- Highlight the things you'd want to bring to the new team.
- Highlight the things you want the new team to teach you.

A template answer

> *"My current team is in the middle of adoption — some folks are heavy users, some are skeptical, no formal policy yet. I've been the unofficial advocate for one small thing: every AI-assisted PR gets a sentence in the description about what the prompt was. It's the smallest possible policy and it's already changed our review conversations. I'd want to bring that habit anywhere I went. What I'd want to learn from a more mature team is the tooling-investment side — system prompts, eval suites, prompt versioning. We don't do any of that yet."*

That answer is honest about your current team, modest about your contribution, and curious about what you'd learn. **All three are likable.**

---

12. The "have you ever fired someone for AI misuse?" question

A wildcard question, but it shows up at staff-and-above interviews.

The right answer

> *"I have not. I have had hard performance conversations rooted in patterns of low-quality AI-generated code — the same conversation you'd have about any low-quality code, with the AI as part of the diagnosis. The fix in every case has been mentorship and review discipline, not termination. I think 'fired for AI misuse' is the wrong frame; the underlying issue is always quality, accountability, or trust. The tool is incidental."*

That answer reframes the question and demonstrates judgment. **You don't take the bait.**

---

13. The cultural pillars worth naming

Throughout cultural questions, three pillars come up repeatedly. Memorize the names so you can refer to them by name.

13.1. Accountability

The human owns every line, regardless of who typed it.

13.2. Transparency

AI use is acknowledged, not hidden.

13.3. Skill stewardship

The team protects the long-term capabilities of its engineers, especially juniors.

When you say things like *"my team's AI culture rests on three pillars: accountability, transparency, and skill stewardship,"* the interviewer hears: this person has thought about this systematically. **It's a high-leverage line.**

---

14. Common pitfalls

14.1. Sounding like a policy memo

If your answers are full of *"the team must…"* and *"developers should…"*, you sound like HR. Use first-person and concrete examples. *"On my last team…"* is more vivid than *"developers should…"*

14.2. Demonizing AI

If every answer is about risks and downsides, you sound like a contrarian who'll slow the team. **Balance with the upside.** AI is a real productivity gain; you respect that.

14.3. Missing the people angle

These are *culture* questions. If your answer is mostly about tooling, you missed the question. **Bring people into every answer** — juniors, seniors, reviewers, managers.

14.4. Pretending you've solved it

If you claim your team has perfect AI culture, the interviewer doesn't believe you. **Acknowledge ongoing work.** *"We're still figuring out X…"* sounds far more credible than *"we have all of this nailed."*

---

15. Practice — the seven-question drill

Set a timer. Answer each of the seven cultural questions in section 2 for 60 seconds, out loud, without notes. Record yourself.

The first pass will be rough. The second will be better. The third will sound real.

You should be able to answer all seven within five working days of practice.

---

16. CHECK YOURSELF

- ☐ Can you list the seven cultural questions from memory?

- ☐ Can you describe a healthy AI-using team and a sick one in concrete terms?
- ☐ Can you handle the "your teammate's PRs are causing tech debt" question with peer-first instincts?
- ☐ Can you deliver the *"AI raises the floor for juniors; our job is to ensure it doesn't lower their ceiling"* line naturally?
- ☐ Can you reframe a challenging cultural question without taking the bait?

---

17. Where you are now

You have the cultural answer set. Combined with the technical answers from Chapters 1–5 and the risk answers from Lesson 6.1, you can navigate any AI-related question that comes up in an interview.

Move on to the closing lesson: **03-portfolio-narrative.md** — your 90-second pitch, the 18 prepared questions catalog, and the closing moves that end the loop strong.